

ZUGAR

Peer-to-Peer Rental Marketplace

Technical Documentation

Design Patterns Lab — Final Project

BSSE, IIT — University of Dhaka, 16th Batch, Group A

Repository: <https://github.com/Afifhasan2003/zugar>

Language & Stack: Java 21 · JavaFX · SQLite · Maven

1. Project Overview

Zugar is a desktop peer-to-peer rental marketplace built with JavaFX and SQLite. It allows users to list personal belongings for rent, discover items listed by others, submit rental requests with custom pricing, negotiate via counter-offers, and manage the full approval lifecycle — all within a local, persistent application.

The domain was chosen because it naturally demands several non-trivial workflows: authentication, multi-state request lifecycle management, owner-side approval and negotiation, and item availability control. These workflows create genuine design problems that justify the application of multiple design patterns beyond simple CRUD.

1.1 Core Entities

Entity	Description
User	Platform participant. Can act as both an item owner and a renter. Identified by UUID, stored with a password hash.
RentalItem	An item listed for rent. Has category, price per day, deposit, condition, location, and an access restriction policy.
RentalRequest	A rental request from a renter to an owner. Tracks offered price, optional counter-price, date range, message, and current status through its lifecycle.
ProjectFile	A user-owned file entity stored in the database. Demonstrates a second service/repository pair independent of the rental domain.

1.2 Application Screens

Screen	Description
Login	Username/password authentication. Token issued on success.
Register	New user account creation with validation.
Catalog	Browse all items listed by other users. Filter by category and keyword search.
Rental Request Dialog	Inline dialog to submit a rental request with custom dates, price, and message.
Requests	Two-panel view: incoming requests (owner side) and sent requests (renter side). Supports Accept, Reject, and Counter-Offer actions.
My Items	Owner dashboard. List own items, toggle availability, delete listings.
Add Item Dialog	Form to publish a new rental listing with all item attributes.

2. Architectural Overview

The application follows a layered architecture with clear separation of responsibilities across four layers:

- Presentation Layer — JavaFX controllers and FXML views. Controllers are thin: they receive user events, delegate to services or commands, and refresh the UI.
- Service Layer — AuthService, ProjectFileService. Encapsulates business rules such as registration validation, password verification, and JWT issuance.
- Repository Layer — Interfaces (RentalRepository, UserRepository, ProjectFileRepository) with SQLite-backed implementations. All SQL is isolated here.
- Infrastructure — DatabaseManager (connection, schema creation, seeding), JwtService (token generation and validation), PasswordHasher (SHA-256 hashing).

Controllers depend on service and repository interfaces, not concrete implementations. This makes the system testable: the test suite wires in a temporary SQLite file and exercises AuthService, JwtService, and ProjectFileService without starting a JavaFX window.

2.1 Package Structure

com.zugar	
ZugarApp.java	– JavaFX Application entry point; wires all dependencies
Main.java	– Thin launcher (required for JavaFX fat-jar packaging)
command/	– Command pattern: Accept, Reject, CounterOffer
controller/	– JavaFX controllers (Login, Register, Catalog, Requests, MyItems, AddItem, MainLayout)
db/	– DatabaseManager: Singleton, schema init, seeder
factory/	– ViewFactory: static factory for FXML loading
model/	– Plain data classes: User, RentalItem, RentalRequest, ProjectFile
repository/	– Repository interfaces + SQLite implementations
service/	– AuthService, JwtService, PasswordHasher, ProjectFileService
state/	– State pattern: RentalRequestState, Pending, Negotiating, Accepted, Rejected

3. Database Design

SQLite is used as the persistent store. The database is initialised on every application launch by `DatabaseManager.initializeDatabase()`, which creates tables if they do not exist and seeds dummy data on first run. Foreign key enforcement is explicitly enabled per connection via `PRAGMA foreign_keys = ON`.

3.1 Schema

users

Column	Definition
id	TEXT PRIMARY KEY (UUID)
username	TEXT UNIQUE NOT NULL
password_hash	TEXT NOT NULL — SHA-256 hex digest
created_at	INTEGER — Unix timestamp, DEFAULT UNIXEPOCH()

rental_items

Column	Definition
id	TEXT PRIMARY KEY (UUID)
owner_id	TEXT NOT NULL — FK → users(id)
title	TEXT NOT NULL
category	TEXT NOT NULL
description	TEXT
price_per_day	REAL NOT NULL
deposit	REAL NOT NULL
condition	TEXT
location	TEXT
access_restriction	TEXT — e.g. "Anyone", "University Members Only"
available	INTEGER DEFAULT 1 — boolean flag

rental_requests

Column	Definition
id	TEXT PRIMARY KEY (UUID)
item_id	TEXT NOT NULL — FK → rental_items(id)
renter_id	TEXT NOT NULL — FK → users(id)

Column	Definition
owner_id	TEXT NOT NULL — FK → users(id)
start_date	TEXT (YYYY-MM-DD)
end_date	TEXT (YYYY-MM-DD)
offered_price	REAL NOT NULL — renter's proposed price per day
counter_price	REAL — owner's counter-offer (nullable)
status	TEXT DEFAULT 'PENDING' — state machine value
message	TEXT — renter's message to owner

project_files

Column	Definition
id	TEXT PRIMARY KEY (UUID)
user_id	TEXT NOT NULL — FK → users(id)
filename	TEXT NOT NULL
content	TEXT NOT NULL
updated_at	INTEGER — Unix timestamp

3.2 Seeder

DatabaseManager.seedDummyData() inserts three users (alice, bob, charlie — all with password "password123") and ten rental items spanning six categories (Photography, Electronics, Audio/Visual, Sports, Vehicles, Other) across different Dhaka locations. Seeding is guarded by a row-count check so it runs only once on a fresh database.

4. Design Patterns

Every pattern in the codebase was introduced to solve a specific design problem. This section documents each pattern: the problem that motivated it, why it was chosen, how it is implemented, and what future benefit the design provides.

4.1 Singleton — DatabaseManager

Problem

SQLite is a file-based database. If multiple parts of the application each create their own connection manager, there is no single place to enforce connection configuration (such as `PRAGMA foreign_keys = ON`) and no centralised control over the database URL. This is especially important because the test suite needs to redirect the database to a temporary file — something that is impossible if the URL is hardcoded in multiple places.

Solution

DatabaseManager uses a thread-safe double-checked locking Singleton. The private constructor prevents direct instantiation. The volatile instance field ensures the fully-constructed object is visible across threads.

```
private static volatile DatabaseManager instance;

public static DatabaseManager getInstance() {
    if (instance == null) {
        synchronized (DatabaseManager.class) {
            if (instance == null) instance = new DatabaseManager();
        }
    }
    return instance;
}
```

Why Not a Static Utility Class?

A static class cannot be injected or mocked. The Singleton instance allows the test suite to call `setDbUrl()` to redirect to a temp file before each test, making integration tests self-contained without modifying production code.

Future Benefit

If the application is later extended to support connection pooling or a different database backend, the change is confined to DatabaseManager alone. No other class references the JDBC URL directly.

4.2 Factory Method — ViewFactory

Problem

Each screen transition in the application requires loading an FXML file, extracting its root node and controller, and wiring up the controller with dependencies. Without a central factory, this boilerplate is duplicated across every class that navigates between screens, and the FXMLLoader is coupled directly to the caller.

Solution

ViewFactory.loadView(String fxmlPath) encapsulates FXML loading and returns a ViewResult holding both the root Parent and the typed controller. Callers extract what they need:

```
ViewFactory.ViewResult res = ViewFactory.loadView("/fxml/catalog.fxml");
CatalogController controller = res.getController();
```

```
controller.init(rentalRepository, currentUser);
mainContainer.setCenter(res.getRoot());
```

Why Static Factory and Not an Interface?

All views in this application are loaded from classpath FXML resources using the same mechanism. A static factory method captures the shared creation logic without the overhead of an abstract creator class. If a second loading strategy were needed (e.g., loading from a remote URL), the method could be overloaded or the class extended.

Future Benefit

Adding a new screen requires only a new FXML file. The factory does not need to change. Navigation logic in `MainLayoutController` calls the same `loadView()` method regardless of which screen is being loaded, keeping navigation code uniform and easy to extend.

4.3 State — Rental Request Lifecycle

Problem

A rental request moves through a lifecycle: `PENDING` → `NEGOTIATING` → `ACCEPTED` or `REJECTED`. At each state, different actions are permitted (e.g., a `REJECTED` request cannot be accepted again). Without the State pattern, this logic would be a chain of if/else or switch statements scattered across controllers and repository methods — fragile and hard to extend if new states are added.

Solution

`RentalRequestState` is an interface with methods: `canAccept()`, `canReject()`, `canCounter()`, `onAccept()`, `onReject()`, `onCounter()`, `getDisplayName()`, and `toStatusString()`. Four concrete classes implement it:

State	Behaviour
PendingState	All three actions allowed. Transitions to Accepted, Rejected, or Negotiating.
NegotiatingState	All three actions still allowed. Counter-offers on an already-negotiating request remain in NegotiatingState.
AcceptedState	Terminal. No actions allowed. All transition methods return this.
RejectedState	Terminal. No actions allowed. All transition methods return this.

`RentalRequestStateFactory.fromStatus(String)` reconstructs the correct state object from the database string. Controllers use the state to decide which buttons to render:

```
RentalRequestState state =
RentalRequestStateFactory.fromStatus(req.getStatus());
if (state.canAccept()) { /* show Accept button */ }
if (state.canCounter()) { /* show Counter field */ }
```

Future Benefit

Adding a new state (e.g., `CANCELLED` or `EXPIRED`) requires only a new class implementing `RentalRequestState` and one line in `RentalRequestStateFactory`. No existing state class or controller needs to change. This is a textbook open/closed principle benefit.

4.4 Command — Owner Request Actions

Problem

The owner can perform three distinct actions on a request: accept, reject, or counter-offer. Each action involves reading the current state, computing the new state, and writing to the repository. Putting this logic directly in the controller creates large event-handler methods that are hard to test and hard to reuse if the same action is triggered from multiple places in the UI.

Solution

Three Command classes encapsulate each action:

Command	Responsibility
AcceptRequestCommand	Reads current state via RentalRequestStateFactory, calls onAccept(), writes new status via rentalRepository.updateRequestStatus().
RejectRequestCommand	Same pattern: reads state, calls onReject(), writes REJECTED status.
CounterOfferCommand	Calls rentalRepository.updateCounterOffer() which atomically sets the counter_price and flips status to NEGOTIATING.

The controller creates and executes the command in a single line, then refreshes the UI:

```
acceptBtn.setOnAction(e -> {  
    new AcceptRequestCommand(rentalRepository, req).execute();  
    loadRequests();  
});
```

Command + State Collaboration

AcceptRequestCommand and RejectRequestCommand delegate to the State pattern to determine the correct next status. This means the Command does not contain any state-transition logic itself — it only orchestrates. This keeps each class focused on a single responsibility.

Future Benefit

Commands can be stored in a queue for undo/redo support. They can be logged for an audit trail. They can be replayed or retried independently of the UI. None of these extensions require changes to the controller.

4.5 Repository — Data Access Abstraction

Problem

Business logic and controllers must not write SQL directly. If they did, switching the database or testing without a real database would require changing every class that touches data. Additionally, queries scattered across the codebase are hard to audit, optimise, or secure against SQL injection.

Solution

Three repository interfaces define the data contract: UserRepository, RentalRepository, ProjectFileRepository. SQLite-backed implementations (UserRepositoryImpl, RentalRepositoryImpl, ProjectFileRepositoryImpl) contain all SQL using PreparedStatements. Callers depend only on the interface:

```
// In ZugarApp.init():  
UserRepository userRepository = new UserRepositoryImpl();  
// AuthService receives the interface, not the impl:
```



```
this.authService = new AuthService(userRepository, jwtService);
```

Future Benefit

The test suite demonstrates this benefit concretely: it wires real implementations against a temp SQLite file. If a future developer wanted to test AuthService with a mock user store, they would simply pass in a different UserRepository implementation. No production code changes.

4.6 Facade — AuthService

Problem

The authentication workflow involves four distinct subsystems: input validation, password hashing (PasswordHasher), user persistence (UserRepository), and token issuance (JwtService). Controllers should not have to orchestrate all four systems directly for every login or registration action.

Solution

AuthService provides a simple interface with three methods: register(), login(), and validateAndGetUser(). Each method coordinates the underlying subsystems internally. The LoginController calls only:

```
String token = authService.login(username, password);  
User user    = authService.validateAndGetUser(token);
```

It has no knowledge of password hashing, JWT signing keys, or repository queries.

Future Benefit

If the hashing algorithm is upgraded from SHA-256 to bcrypt, only PasswordHasher changes. If the token format changes from JWT to a session cookie, only JwtService changes. AuthService coordinates; it does not implement any of these concerns itself.

4.7 Strategy — PasswordHasher

Problem

Password hashing is a security-sensitive operation that should be replaceable independently of the authentication flow. Hardcoding SHA-256 calls inside AuthService would make it impossible to swap the algorithm without modifying authentication business logic.

Solution

PasswordHasher is a dedicated class that encapsulates the hashing algorithm (currently SHA-256). AuthService calls PasswordHasher.hashPassword() and PasswordHasher.verifyPassword() — it does not know which algorithm is used. JwtService is similarly isolated: it handles only token creation and validation, with its own secret key and expiry configuration.

Future Benefit

Upgrading to bcrypt requires only a new implementation of PasswordHasher's two static methods. AuthService and all tests remain unchanged. The isolation also makes it straightforward to add a password strength validator without touching the hashing logic.

5. Key Workflows

5.1 User Registration and Authentication

Step	Detail
1. User fills Register form	RegisterController calls authService.register(username, password)
2. Validation	AuthService checks for empty fields and duplicate username via UserRepository
3. Password hashing	PasswordHasher.hashPassword() produces a SHA-256 hex digest
4. Persistence	UserRepositoryImpl.save() inserts into the users table
5. Login	authService.login() verifies password hash and calls jwtService.generateToken()
6. Token validation	authService.validateAndGetUser() parses JWT, extracts userId, fetches User from DB
7. Navigation	ZugarApp.showMainLayout(user) wires MainLayoutController with the authenticated User object

5.2 Rental Request Lifecycle

Step	Detail
1. Renter browses catalog	CatalogController loads all items, filters by search query and category, excludes own items
2. Renter submits request	Dialog collects dates, offered price, message — RentalRequest saved with status PENDING
3. Owner views incoming	RequestsController loads requests where owner_id = currentUser.id
4. Owner accepts	AcceptRequestCommand → State.onAccept() → status becomes ACCEPTED
5. Owner rejects	RejectRequestCommand → State.onReject() → status becomes REJECTED
6. Owner counters	CounterOfferCommand → updateCounterOffer() → counter_price set, status becomes NEGOTIATING
7. Renter views counter	Sent requests panel shows counter_price; renter can Accept counter-offer or counter back
8. Resolution	Request reaches terminal state: ACCEPTED or REJECTED

5.3 Item Management

Action	Implementation
List items	MyItemsController shows only items where owner_id = currentUser.id
Publish listing	AddItemController form → RentalRepositoryImpl.saveItem() with UUID primary key
Toggle availability	RentalRepositoryImpl.updateItem() flips the available flag
Delete listing	RentalRepositoryImpl.deleteItem() removes the row by id

6. Test Cases

ZugarAppTest.java contains JUnit 5 tests that run against a temporary SQLite database created in a @TempDir, ensuring tests are isolated and do not affect the production database. The Singleton's setDbUrl() method enables this redirection without any mocking framework.

Test Method	What It Verifies
testDatabaseTablesCreated()	Verifies that initializeDatabase() creates the users and project_files tables in SQLite.
testUserRegistrationAndLogin()	Registers a user, logs in, validates the returned JWT, and confirms the extracted User matches the registered one.
testInvalidLoginThrowsException()	Confirms that an incorrect password throws IllegalArgumentException with the expected message.
testProjectFileRepositoryAndService()	Full CRUD cycle: create two files, list them, update one, delete the other, confirm remaining count.
testJwtTokenParsingAndValidation()	Generates a token and verifies that extractUserId and extractUsername return the correct values.

7. Pattern Summary

Pattern	Where & Why
Singleton	DatabaseManager — single database connection manager; testable via setDbUrl()
Factory Method	ViewFactory — centralised FXML loading; decouples navigation from FXMLLoader
State	RentalRequestState hierarchy — models PENDING → NEGOTIATING → ACCEPTED/REJECTED lifecycle
Command	AcceptRequestCommand, RejectRequestCommand, CounterOfferCommand — encapsulates owner actions
Repository	UserRepository, RentalRepository, ProjectFileRepository interfaces — isolates all SQL
Facade	AuthService — simplifies login/register workflow for controllers; hides hashing and JWT internals
Strategy	PasswordHasher — isolated, replaceable hashing algorithm; JwtService similarly isolated

8. Project Requirement Checklist

Requirement	Status
JavaFX desktop application	✓ JavaFX 21 with FXML and CSS styling
Maven project management	✓ pom.xml with all dependencies declared
SQLite persistent database	✓ zugar.db, survives application restarts
4+ meaningful database entities	✓ users, rental_items, rental_requests, project_files
CRUD for 3+ entities	✓ Users (register/find), RentalItems (create/read/update/delete), RentalRequests (create/read/update status + counter)
2+ multi-step workflows	✓ Authentication flow (register → hash → persist → JWT); Rental lifecycle (request → negotiate → accept/reject)
Search and filtering	✓ Catalog filtered by keyword (title + description) and category; repository queries by owner and renter
4–8 major screens	✓ Login, Register, Catalog, Requests, My Items, Add Item dialog, Rental Request dialog
Database seeders	✓ DatabaseManager.seedDummyData() — 3 users, 10 items across 6 categories
Design patterns applied	✓ Singleton, Factory Method, State, Command, Repository, Facade, Strategy
Test cases for business logic	✓ 5 JUnit 5 tests covering auth, JWT, CRUD, and DB init
Appropriate separation of layers	✓ Presentation / Service / Repository / Infrastructure clearly separated
Foreign keys and constraints	✓ PRAGMA foreign_keys = ON; FK from items → users, requests → items/users